+ **Lecture No. :** +

■ **Theory Content to be taught :**

1. Unit 2 : stack, Queue.
2. stack, ADT stack and it's operations.
3. Algorithms and their complexity analysis.
4.
5.

■ **Specific Objectives :**

1. Outline concepts of stack and queue to solve
2. real-time problems.
3.
4.
5.

■ **Theory Content :**

★ Introduction to stack :-

A stack is a ~~most~~ one of the most essential non-primitive data structure. Implementation of most of the system program is based on stack data structure.

The stack data structure follows the principle of "Last-In-First-Out" (LIFO) in which the last item pushed (inserted) on to the stack is always the first to

to be removed from the stack. The insertion and deletion operations are known as "PUSH" & "POP" respectively.

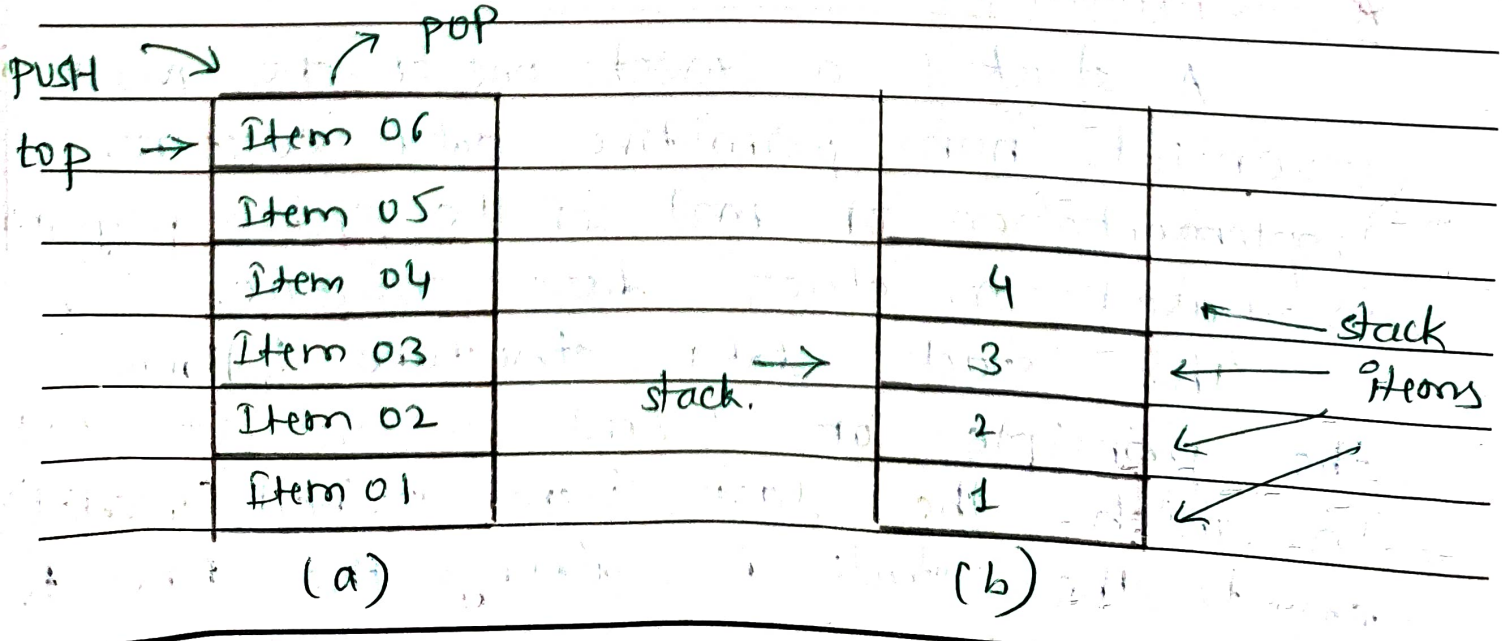
There are several applications like recursive function call, evaluation of expression etc where a stack is mainly used.

Example - Browser back button, undo operation in text editors, function call stack in programming

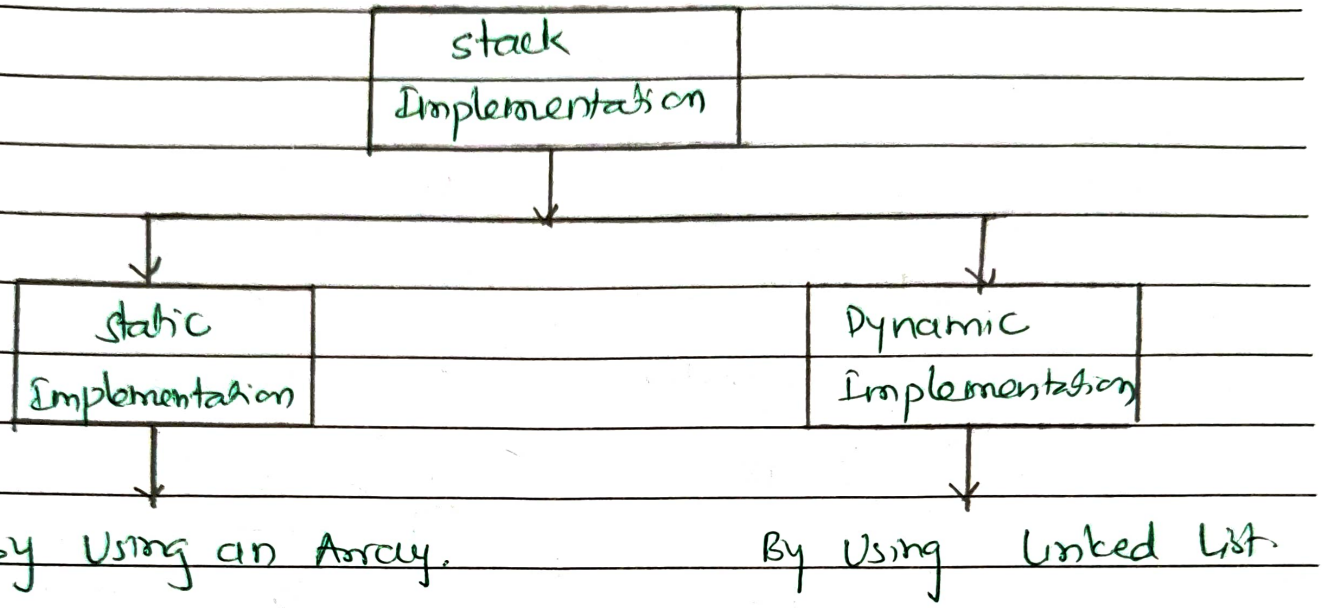
- **Stack Representation :-**

A stack is an ordered list with the restriction that elements are added or deleted from only one end of the list termed "top" of stack.

The insertion of element on the stack is called as "PUSH" and deletion operation is called as "pop". i.e. when an element is added to the stack the operation is called PUSH and when it is removed the operation is called POP.



The representation of stack in the memory can be done in two ways —



- **stack ADT (Abstract Data Type) :-**

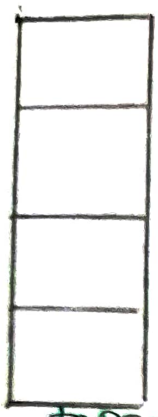
A stack ADT defines what operations can be performed on a stack, without specifying how they are implemented.

- **Basic operations :-**

Operation	Description
push(x)	Insert element x at the top.
pop()	Remove & return top element.
isEmpty()	Check whether stack is empty.
isFull()	Check whether stack is full.

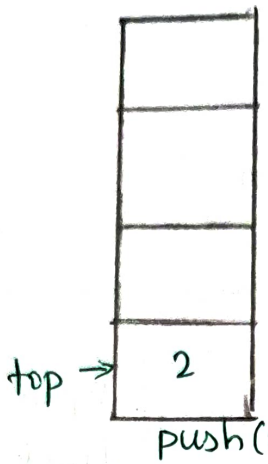
- PUSH and POP operation example :-

① PUSH Operation :-

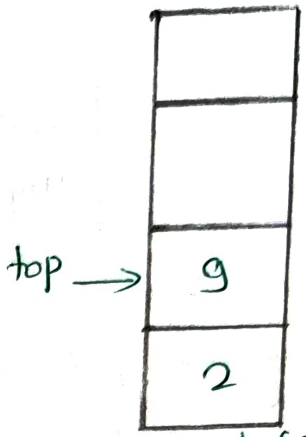


empty stack.

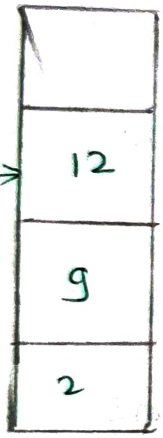
top = -1.



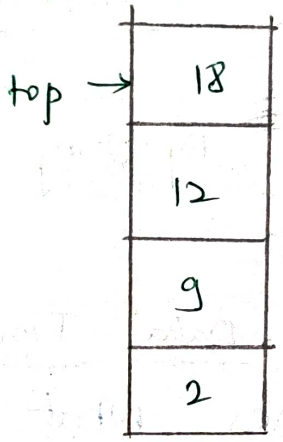
push(2)



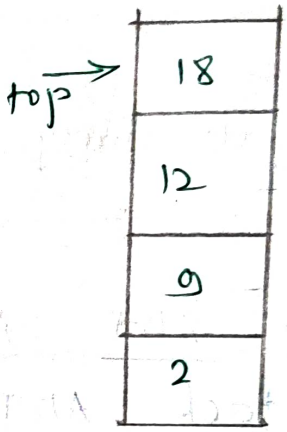
push(9)



push(12)



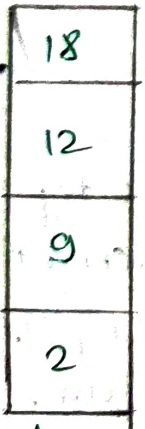
push(18)



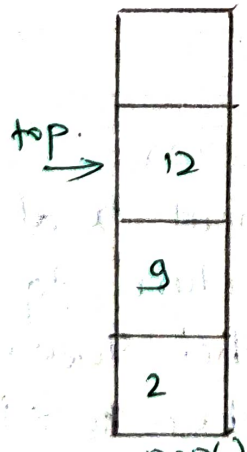
push(20)

"stack" overflow condition.

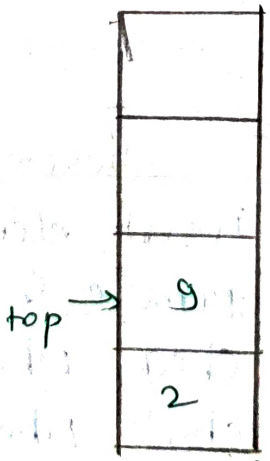
② POP Operation :-



stack initially ⇒ pop()

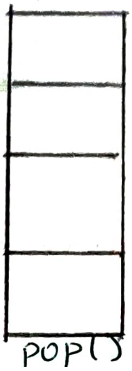


pop()



pop()

pop



pop().

top = -1.

stack underflow condition.

⓪ stack operations :-

A stack is a linear data structure in which addition of new element or deletion of an element occures only at one end. This end is called "top" of the stack.

The operation of adding new element in the stack and deleting an element from the stack is called PUSH and POP respectively.

(a) PUSH operation :-

The process of inserting a new element to the top of stack is called PUSH operation.

When a new element is inserted to the stack, the top is incremented by one.

In case, if stack is full, then no element can be inserted into the stack, it is called 'stack full' condition or 'stack overflow' condition.

- Algorithm :- Let a[size] is an array.

step 1 : if (top == size-1) then // stack full cond?
print "stack overflow".

step 2 : else
accept 'item' to be inserted into stack.

step 3 : set top = top + 1.

step 4 : a[top] = item.

• Complexity analysis of push operation :-

Time Complexity :

During push operation : 1. check stack full.
2. increment top
3. Insert element.

Each step takes constant time.

$T(n) = c1 + c2 + c3$. (c_1, c_2, c_3 - constant)

Therefore, $T(n) = O(1)$.

Best case: $O(1)$ - inserted immediately.
Average case: $O(1)$ - same operation performed.
Worst case: $O(1)$ - if stack is full, same no. of steps.

(b) Pop operation :-

The process of deleting an element from the top of the stack is called 'pop' operation. After every 'pop' operation 'top' is decremented by one. If there is a no element in the stack and the pop operation is performed then this will result into 'stack underflow' or 'stack empty' condition.

• Algorithm :- Let a[size] is a stack. array.

step 1 : If (top == -1) then // stack empty condn.
print 'stack is empty'

step 2 : else
item = a[top].

step 3 : top = top - 1.

step 4 : print deleted 'item'.

Complexity analysis of pop operation:-

Time Complexity :-

Operation performed - 1. Empty stack.
2. Read top element.
3. decrement top.

Number of operations does not depend on stack size. $T(n) = c_1 + c_2 + c_3$.

Therefore, $T(n) = O(1)$.

Best case : $O(1)$

Average Case : $O(1)$

Worst Case : $O(1)$.

→ Why all stack operations are $O(1)$?.

In stack operation -

(a) We always work on the top.

(b) No traversal required.

(c) No searching is required.

(d) No shifting of element required.

whether stack contains any no. of elements.

Shri Shamrao Patil (Yadravkar) Educational & Charitable Trust's
Sharad Institute of Technology College of Engineering
YADRAV (Ichalkaranji) - 416 101. Tal. Shirol, Dist. Kolhapur (M.S.) India
+ Approved by AICTE, New Delhi + Recognized by Government of Maharashtra + Affiliated to DBATU, Lonere

+ **Lecture No. :** +

■ **Theory Content to be taught :**

1. Applications of stack : Expression conversion
2. and evaluation - corosponding algorithms
3. and it's complexity analysis.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

❶ Application of stack : Expression conversion & evaluation

An expression is a combination of operands and operators.

fx :- A+B, (A+B)*C, 5+6*2.

Computers can not directly evaluate complex infix expressions efficiently. Therefore expressions are converted into other forms using stack.

◎ Expression Notations :-

A) Infix Expression :-
Operator appears between operands.
Example; A+B, A*B+C.

B) prefix Expression :-
Operator comes before operands.
Example : +AB, *+ABC.

C) postfix Expression :-
Operator comes after operands.
Example : AB+, AB*C+.

◎ Operator procedence :-
The order of evalution is carried out according to the priority of arithmetic operators.

sr.No.	operator	priority
1.	Exponential (^)	High
2.	multiplication (*), division (/)	mid
3.	addition (+), substraction (-)	low.

The operator of the same priority are performed from left to right. While the operators contained within parenthesis are evaluated first. The exponential (^) operators are evaluated from right to left.

Conversion of infix to postfix & evaluation of postfix expression.

To reduce the complexity of expression evaluation (infix), the postfix or prefix expressions are used in the computer programs.

~~Algorithm~~ → steps :-

1. Read the infix expression from left to right.
2. If the token is an operand (alphabet or digit), add it directly to the postfix expression.
3. If the token is an opening bracket (, push it onto the stack.
4. If the token is a closing bracket), pop & output all operators from the stack until the corresponding opening bracket (is found. Remove the opening bracket from the stack.
5. If the token is an operator :
   (a) If the stack is empty, push the operator on to the stack.
   (b) otherwise, compare it's precedence with the operator at the top of the stack.
   (c) If the incoming operator has higher precedence, push it onto the stack.
   (d) Otherwise, pop operators from the stack and add them to the postfix expression until a lower-precedence operator or an opening bracket is found. Then push the incoming operator onto the stack.
6. Repeat steps 2 to 5 until all tokens of the ~~stack~~ infix expression are processed.

7. pop all remaining operators from the stack and add them to the postfix expression.

Example :- A+B*C

Token	stack	postfix Expression
A	empty	A
+	+	A
B	+	AB
*	+, *	AB
C	+, *	ABC
-	+	ABC*
-	empty	ABC*+-

Note :- No two operators of same priority can stay together in the stack.

- Evaluation of postfix Expression :-

1. Read the postfix expression from left to right.
2. Read the next element (token).
3. If the element is an operand, push it into the stack.
4. If the element is an operator
   (a) pop the top operand from the stack and store it as S1.

(b) pop the next operand from the stack and store it as S2.

(c) perform the operation (S2 operator S1).

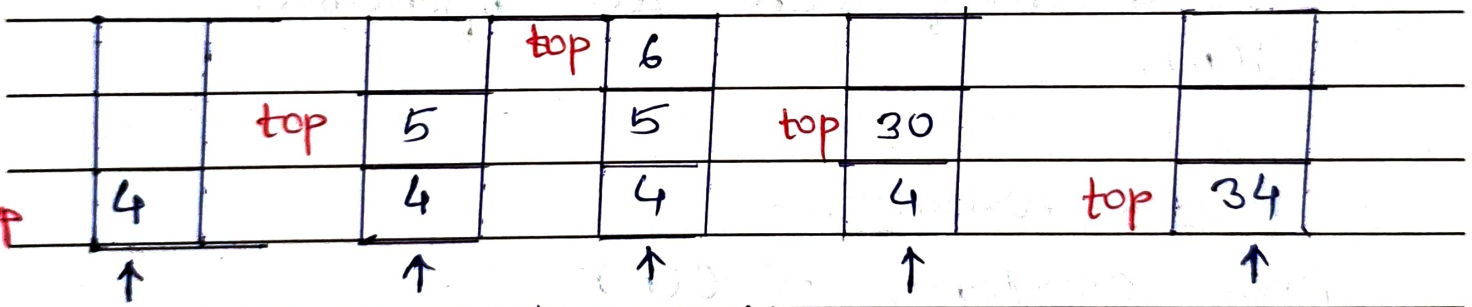
(d) push the result back onto the stack.

5. Repeat the steps 2 to 4 until all elements of the postfix expression are processed.

6. After the entire expression has been scanned, the stack will contain only one element, which is the final result.

7. pop and return the final result.

Example:- 4 5 6 * +



push(4) push(5) push(6) pop twice & do math & push the result 5 * 6 = 30. push(30) pop twice & do math & push the result. 4 + 30 = 34 push(34).

Complexity analysis of infix to postfix Conversion:

Algorithm Overview:

1. scan expression from left to right.
2. operands are added to output.
3. operators are pushed onto stack.
4. Operators are popped according to precedence.
5. Remaining operators are popped at the end.

Let 'n' is the no. of symbols

Time Complexity -

Each symbol pushed/popped at most once.

$O(n)$.

Space Complexity -

stack may store all operators.

$O(n)$.

- postfix evaluation:-

Time complexity = $O(n)$.

space complexity = $O(n)$.

+ Lecture No. : +

■ **Theory Content to be taught :**

1. Conversion to infix to prefix -
2.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

● Conversion of Infix to Prefix :

We learned how to convert infix to postfix expression. We use the same to convert infix to prefix.

1. Reverse the Infix expression
   → While reversing, interchange the parenthesis.
   - Replace every '(' with ')'.
   - Replace every ')' with '('

2. Convert the modified infix expression into postfix.
   → Apply the standard infix-to-postfix conversion algorithm to the modified expression obtained in step 1.
   → This will produce the postfix form of the modified expression.

3. Reverse the postfix expression.
   → Reverse the postfix expression obtained in step 2.
   → The resulting expression is the prefix expression.

Example –

Infix ⇒ (A+B) * C

1. Reverse & change parentheses.
   C * (B+A).

2. Convert to postfix ⇒ CBA + *

3. Reverse postfix ⇒ * + ABC

prefix ⇒ * + ABC.

ⓐ Evaluation of Prefix Expression:

1. Scan the prefix expression from Right to left.

2. If the symbol is an operand -
   → push the operand onto the stack.

3. If the symbol is an operator -
   → pop the first operand from the stack and store in S1.
   → pop the second operand from the stack and store in S2.
   → perform the operation (S2 operator S1).
   → push the result back onto the stack.

4. Repeat until the entire expression is processed.

5. Obtain the final result.
   → After the complete expression has been processed, the stack will contain only one element.
   → pop this element from the stack. This is the final result of the prefix expression.

+ **Lecture No. :** +

■ **Theory Content to be taught :**

1. Recursion - factorial, GCD, Fibonacci
2. sequence, Tower of Hanoi.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

◎ Recursion :-

stack data structure used for implementing recursion. Recursion is a method of problem solving.

Recursion occurs when function calls itself directly or indirectly. It make coding programs or solutions much faster and simpler.

Recursion is used when large problems can be broken down into smaller, more managable subproblems.

In recursion function is run repeatedly on smaller inputs until base case or smaller input with simple answer is reached.

Example- factorial of a given no.

factorial (4).

Execution ⇒

Factorial (4)
= 4 × factorial (3).
= 4 × 3 × factorial (2)
= 4 × 3 × 2 × factorial (1)
= 4 × 3 × 2 × 1 × ~~factorial (0)~~
= 4 × 3 × 2 × 1 × 1.
= 24.

- How stack is used in Recursion.

Whenever a recursive function is called:
→ The system creates an record (stack frame).
→ The frame stores -
  - function parameters
  - Local variables
  - Return address
  - Temp. data.

This frame is pushed onto the call stack.

When the function completes execution:

- It's stack frame is popped from the stack.
- Control returns to the calling function.

Thus, recursion is implemented internally using a stack.

- **factorial using Recursion:**

Factorial of a number 'n' is:

$n! = n \times (n-1) \times (n-2) \times \cdots \times 1$

Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

$4! = 4 \times 3 \times 2 \times 1 = 24$.

$0! = 1$.

Recursive formula:

$fact(n) = n * fact(n-1)$.

Algorithm :-

1. start
2. Read the value of 'n'.
3. call factorial (n).
4. In factorial (n) :
   - → if n==0 or n==1, return 1.
   - → otherwise return n*factorial (n-1).
5. Display the factorial value
6. stop.

- GCD (Greatest Common Divisor) using recursion:

GCD, also called HCF (Highest common factor), is the largest positive integer that divides two or more numbers exactly without leaving a reminder.

Example:

Find the GCD of 12 and 18.

→ factors of 12 = 1, 2, 3, 4, 6, 12.

factors of 18 = 1, 2, 3, 6, 9, 18.

Common factors : 1, 2, 3, 6.

Greatest Common factor = GCD(12, 18) = 6

Euclid's algorithm for GCD:

Rule :- For two number 'a' and 'b'

GCD (a, b) = GCD (b, a%b).

Repeat until -

b = 0.

Then : GCD (a, 0) = a.

Algorithm :

1. Start
2. Read two integers a and b.
3. Define recursive function gcd(a,b).
4. if b==0
   - Return a.
5. otherwise
   - Return gcd(b, a%b).
6. Call the function.
7. Display the GCD.
8. Stop.

'C' program :-

```
#include <stdio.h>
int gcd(int a, int b)
{
    if (b==0)
        return a;          // base case.
    return gcd(b, a%b);    // recursive call.
}
int main()
{   int a,b;
    printf("enter two numbers");
    scanf("%d%d", &a, &b);
    printf("GCD of %d and %d = %d", a, b,
                                gcd(a,b));
    return 0;
}
```

◉ Fibonacci series using Recursion :-

The Fibonacci series is a sequence of numbers in which each number is obtained by adding the two preceding numbers.
The series starts with -

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Rule:

→ first term = 0
→ second term = 1.
→ every next term = sum of previous two terms.

Mathematically -

$$F(n) = F(n-1) + f(n-2).$$

Where - $f(0) = 0$, $f(1) = 1$.

fibonacci using recursion -

Recursive defination -

$$fib(n) = fib(n-1) + fib(n-2)$$

Base case -

$$fib(0) = 0$$
$$fib(1) = 1.$$

◉ Algorithm :-

1. start
2. Read the no. of terms 'n'.
3. Define recursive function fib(x).
4. If x=0, return 0.
5. If x=1, return 1.
6. otherwise return fib(x-1) + fib(x-2).

7. use loop from 0 to n-1.
8. print fib(i) for each term
9. stop.

'C' program -

```
#include <stdio.h>
int fib (int n)
{
    if (n==0)
        return 0;
    if (n==1)
        return 1;
    return fib(n-1) + fib(n-2);
}
void main()
{
    int n, i;
    printf ("enter no. of terms :");
    scanf ("%d", &n);
    printf ("Fibonacci series :");
    for (i=0; i<n; i++)
    {
        printf (" %d", fib(i));
    }
    getch();
}
```

+ Lecture No. : +

■ **Theory Content to be taught :**

1. Recursion - factorial, GCD, fibonacci sequence,
2. Tower of Hanoi.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

→ C program to find the factorial of a number using a stack (array implementation).

```
#include <stdio.h>
int stk[100], top=-1, n, i, fact=1;

void push(int x)
{
    stk[++top] = x;
}
```

```
int pop()
{
    return stk[top--];
}

int main()
{
    printf("Enter a number");
    scanf("%d", &n);

    // push numbers onto the stack.

    for(i=1; i<=n; i++)
    {
        push(i);
    }

    // pop numbers & calculate factorial.

    while(top != -1)
    {
        fact = fact * pop();
    }
    printf("Factorial = %d", fact);

    return 0;
}
```

⇒ 'C' program to find the GCD (Greatest Common Divisor) using stack.

```
#include <stdio.h>
int stk [100], top = -1;

void push (int x)
{
    stk [++top] = x;
}

int pop ()
{
    return stk [top--];
}

int main()
{
    int a, b, temp;
    printf (" Enter two numbers ");
    scanf ("%d %d", &a, &b);

    push(a);
    push(b);

    b = pop();
    a = pop();
```

```
while (b != 0)
{
    temp = a;
    b = a % b;
    a = temp;
}

printf("GCD = %d", a);

return 0;
}
```

+ Lecture No. : +

■ **Theory Content to be taught :**

1. Recursion - Tower of Hanoi.
2.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

● Tower of Hanoi :-

The tower of Hanoi is a classic mathematical puzzle involving three rods (A, B and C). and 'n' disks of different sizes.

Initially, all disks are stacked on rod 'A' in decreasing order of diameter the largest disk at the bottom and the smallest at the top.

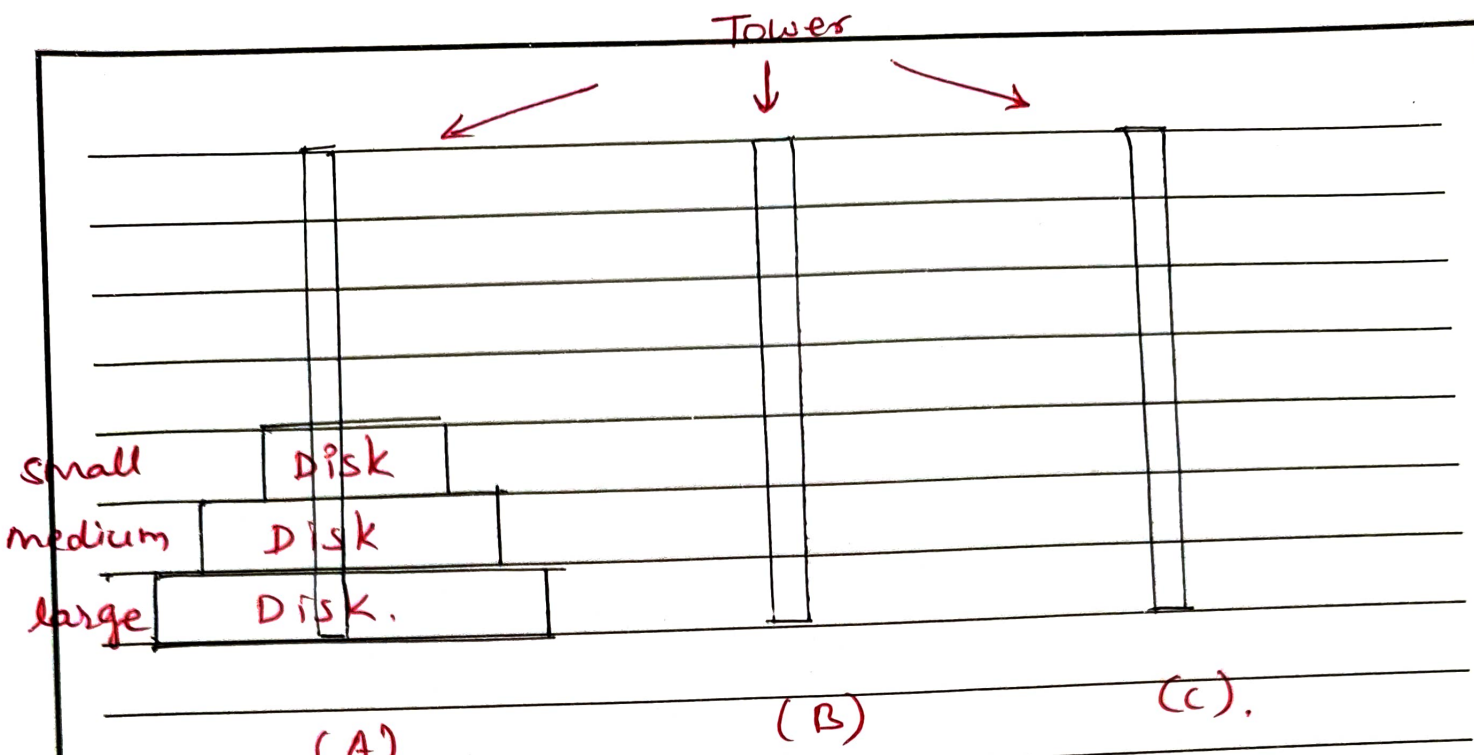
Goal is to move the entire stack to another rod (rod c) while following these rules:

→ move only one disk at a time.
→ At each step, you can take the top disk from any rod and place it on another rod.
→ A disk can only be moved if it is the topmost disk of a rod.
→ No large disk may be placed on top of a smaller disk.

Example :-

Input : n = 3
output :
Disk 1 moved from A to C
Disk 2 moved from A to B.
Disk 1 moved from C to B
Disk 3 moved from A to C
Disk 1 moved from B to A.
Disk 2 moved from B to C.
Disk 1 moved from A to C.

* Tower of Hanoi puzzle with 'n' disks can be solved in minimum $2^n - 1$ steps.


Tower
small
Disk
medium
Disk
large
Disk.
(A)
(B)
(C).

+ **Lecture No. :** _____ +

■ **Theory Content to be taught :**

1. Queue : Simple Queue. / Linear Queue.
2.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

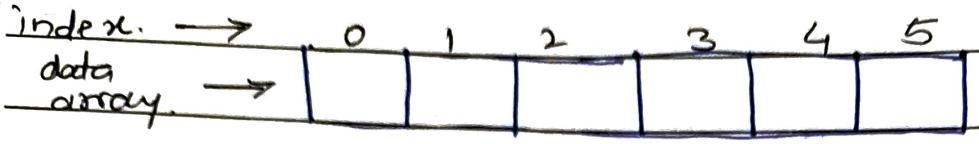
■ **Theory Content :**

Queue :-

A Queue is a linear list of elements in which deletions can take place at one end called front end and insertion can take place only at other end called rear end.

Queue is also called FIFO (first-in-first-out) lists, since the first element in a queue will the first element out of a queue.

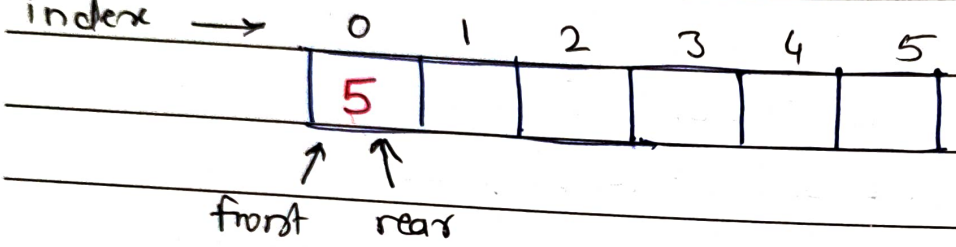
During the initialization of a queue, its front end and rear end are set to -1.



front = rear = -1.
size = 06.

Empty Queue.

Below fig. shows the status of queue after insertion of the element '5'.



front = rear = 0

on subsequent insertions, front remains at the same place, where rear advances.

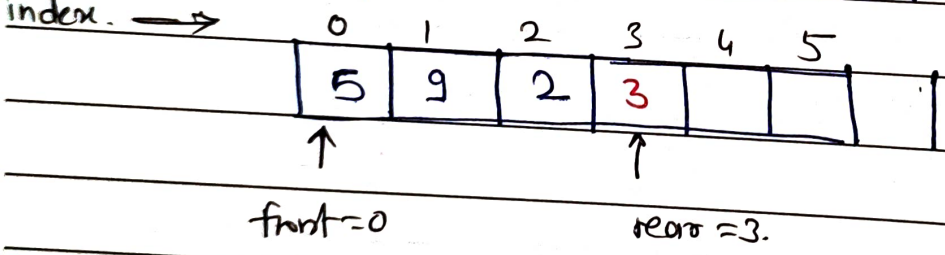


fig. Above fig. shows queue after inserting 4 four elements.

Whenever we insert any element in queue, the value of rear is incremented by one.
i.e. rear = rear + 1.

NOTE:- During the insertion of first element in the queue, we always increment the front by one. i.e. front = front + 1.
Afterwards, front will not changed during the entire insertion operation.

Queue as an ADT :-

The queue operations are given below -

① Enqueue / Insert :- Adds a new item to the rear of the queue.

② Dequeue / delete :- Removes the front item from the queue.

③ queue :- creates a new queue that is empty.

④ isEmpty :- Tests to see whether the queue is empty.

⑤ isFull :- Tests to see whether the queue is full.

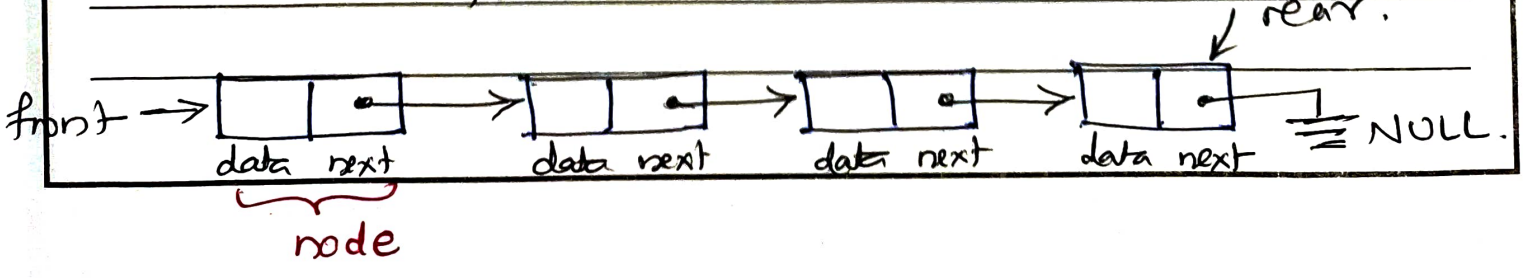
⑥ size / display :- Returns the no. of items in the queue.

Types of queue :-

① simple Queue / Linear Queue :-

A queue is an ordered list in which all insertions take place at one end called rear while all deletions take place at the other end, called front.

Representation of linear Queue using array & linked li



Operations on Linear Queue :-

① **create :-** creates a new queue.

② **add / insert / enqueue :-** Add a new element to the queue. A new element can be added to the queue at the rear.

Algorithm :-

Step 1: check the queue full / overflow condition.
If (rear >= size-1).
then print "Queue is full".
return.

Step 2: if (front == -1) (only for first element)
then set front = 0.

Step 3: set rear = rear + 1.

Step 4: set q[rear] = item.

// insert element at rear position of queue

Note :- When (front == rear), then there are no element in the queue and it is said to be an empty queue.

③ delete / dequeue :-

Removes an element from the queue. This operation removes the element which is at the front of the queue. This operation can only be performed if the queue is not empty.

Algorithm :-

step 1 : if (front == -1)
then print "Queue is empty".

step 2 : else
// delete the element from the queue.
set front = front + 1.

step 3 : if (front > rear)
then
set both front & rear to -1.

④ display :- To display the elements from the queue.

⑤ Queue Full :-

A queue is full when it's rear pointer points to (size - 1). 'size' is a max. number of elements in a queue.

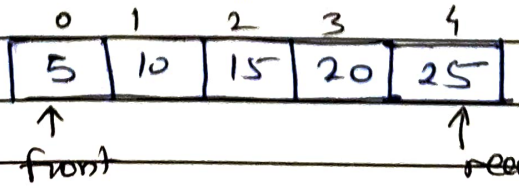
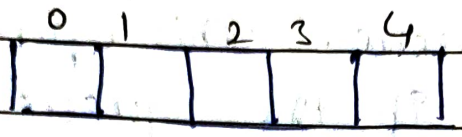


fig: Queue full condition.

⑥ Queue Empty :-

A queue is empty when its front pointer points to -1 position.



front = rear = -1.

fig. empty Queue.

- Complexity Analysis :-

operation	Best case	Average case	Worst case
enqueue	O(1)	O(1)	O(1)
dequeue	O(1)	O(1)	O(1)
display	O(1)	O(1)	O(1).

Space Complexity = O(n).

- Applications :-

① operating system Task scheduling.
② print spooling.
③ All type of customer serve (like booking).
④ Data buffers (N/w & I/O)

● Disadvantage of Linear Queue :-

The main disadvantage of linear queue implemented using array is that when elements are deleted from the queue, new elements can not be added in their place in queue. i.e. position can not be reused.

The queue may appear to be full however there may be space in the queue.

Lecture No. :

■ **Theory Content to be taught :**

1. Queue : Circular Queue.
2. priority Queue.
3.
4.
5.

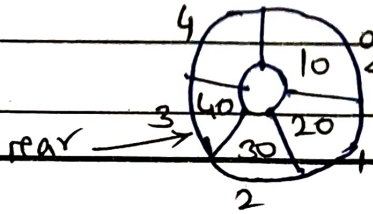
■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

② Circular Queue :-

Circular Queue is a linear data structure in which operations are performed based on FIFO principle and the last position is connected back to the first position to make a circle as shown in the following fig. It is also called ring buffer.



The advantage of circular queue is that, the overflow occurrences will be very minimal. The actual queue content will move around in the ring in a anticlockwise direction.

◉ operations on Linear Queue :-

① create :- creates a new queue.

② Enqueue / Insert :-

There is a formula which has to applied for setting the rear pointer for circular queue. rear = (rear + 1) % size

Algorithm :-

step 1 : check overflow condition.

if ((front == rear + 1) || (front == 0 && rear == size-1))

print 'Queue overflow'.

step 2 : else

if (front == -1)

front = 0

rear = (rear + 1) % size

a[rear] = element.

③ dequeue / delete :-

There is a formula which has to applied for setting the front pointer for circular queue.

front = (front + 1) % size.

Algorithm :-

step 1: check queue overflow condition.

if (front == -1)

then print 'Queue Overflow'.

step 2: else

if (front == rear)

set front = rear = -1

else

front = (front + 1) % size.

④ display :-

To display the elements from the Queue.

⑤ Queue full / Queue Overflow :-

To check whether queue is full or not.

⑥ Queue empty :-

To check whether queue is empty or not.

Complexity analysis :-

operation	Best case	Average case	Worst case
enqueue	O(1)	O(1)	O(1)
dequeue	O(1)	O(1)	O(1)
display	O(1)	O(n)	O(n)

Space complexity = O(n).

③ Priority Queue :-

A priority queue is a special type of queue in which each element is assigned a priority.
- Elements are removed according to their priority in priority queue.
- Higher priority elements are served before lower-priority elements.
- If two elements have the same priority, they are usually served in FIFO order.

Defination: A priority queue is an abstract data type where -
- ✓ every element has a priority value.
- ✓ Deletion returns the element with highest (or lowest) priority.

④ Types of Priority Queue :-

① Max priority Queue :

Highest priority element is removed first.

Ex :
elements - (10, p=2), (20, p=5), (30, p=1)

Deletion order - 20 → 10 → 30.

because - 5 > 2 > 1.

② Min priority Queue :-

Lowest priority element is removed first.

Ex : elements - (10, p=2), (20, p=5), (30, p=1)

Deletion order - 30 → 10 → 20.

because - 1 < 2 < 5.

Priority Queue Representation:

Each element stores - (Data, Priority)

Example - [(A,3), (B,5), (C,1), (D,4)].

priority values -

A = 3 C = 1

B = 5 D = 4.

Operations on Priority Queue :-

① Insert / enqueue :-

Adds an element with it's priority.

Ex : Insert (A,3), (B,5), (C,1).

Queue -

[(B,5), (A,3), (C,1)].

Sorted according to priority.

② Delete / dequeue :-

Removes highest - priority element.

Ex :-

Queue - [(B,5), (A,3), (C,1)].

Delete - (B,5).

Remaining - [(A,3), (C,1)].

③ Is Empty :-

Checks whether queue contains elements.

⊙ Algorithm Complexity analysis :-

operation	Complexity.
Insert	O(1)
delete	O(n). // need to search high priority.

For sorted array :-

operation	Complexity.
Insert	O(n) // need correct positi
delete	O(1).

⊙ Applications of priority Queue :-

1) CPU scheduling
2) printer management
3) Hospital emergency sequence.
4) network routing.

+ **Lecture No. :** +

■ **Theory Content to be taught :**

1. C program to implement -
2. a) Linear queue using array.
3. b) circular queue using array.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

● Write a C program to implement Linear Queue using an array.

```
#include <stdio.h>
#include <stdlib.h>
int q[100], size, i, ch, front=-1, rear=-1;
void enqueue()
{
    int item;
```

```
if (rear >=size-1)
{
    printf ("Queue is Full");
}
else
{
    printf ("enter element");
    scanf ("%d", &item);
    if (front == -1 && rear == -1)
        front = rear = 0 ;
    else
        rear = rear + 1;
    q[rear] = item;
}
}

void dequeue()
{
    int a;
    if (front == -1 || front > rear)
    {
        printf ("Queue is empty");
    }
    else
    {
        a = q[front];
        front = front + 1;
        printf ("deleted element = %d", a);
        if (front > rear)
        {
            front = rear = -1;
        }
    }
}
```

```
void display()
{
    if ((front==-1) || (front>rear))
    {
        printf (" Queue is empty");
    }
    else
    {
        for (i=front; i<=rear; i++)
        {
            printf ("%d\t", q[i]);
        }
    }
}

int main()
{
    printf ("\n Enter size of Queue");
    scanf ("%d", &size);
do{
    printf (" 1. Enqueue\t 2.dequeue\t 3.display\
            4. exit \n");
    printf (" Enter your choice");
    scanf ("%d", &ch);
    switch (ch)
    {
        case 1: enqueue ();
                break;
        case 2: dequeue ();
                break;
        case 3: display ();
                break;
```

```
        case 4: exit(0);
        default: printf("wrong choice");
    }
  }
  while (ch!=4);
  return 0;
}
```

- Write a 'C' program to implement Circular Queue using array.

```
#include <stdio.h>
#include <stdlib.h>

int size, q[100], front = -1, rear = -1;
int ch, x, i;
void enqueue()
{
    int x;
    if ((rear+1) % size == front)
    {
        printf("Queue is full");
        return;
    }
    printf("enter element");
    scanf("%d", &x);
    if (front == -1)
        front = 0;
    rear = (rear+1) % size;
    q[rear] = x;
```

```
void dequeue()
{
    if (front == -1)
    {
        printf ("Queue is empty");
        return;
    }

    printf ("deleted element = %d", q[front]);
    if (front == rear)
        front = rear = -1;
    else
        front = (front + 1) % size;
}

void display()
{
    int i;
    if (front == -1)
    {
        printf ("Queue is empty");
        return;
    }

    i = front;
    while (1)
    {   printf ("%d", q[i]);
        if (i == rear)
            break;
        i = (i+1) % size;
    }
    printf ("\n");
}
```

```
int main()
{
    printf("enter size of queue");
    scanf("%d", &size);
  while(1)
  {
    printf("\n1. Enqueue");
    printf("\n2. dequeue");
    printf("\n3. display");
    printf("\n4. exit");
    printf("\n Enter your choice");
    scanf("%d", &ch);
    switch(ch)
    {
      case 1: enqueue();
              break;
      case 2: dequeue();
              break;
      case 3: display();
              break;
      case 4: exit(0);
    } default : printf("Wrong choice");
  }
  return 0;
}
```